

hardcaml__asic Architecture Direction

Tiny Tapeout First, Extensible Toward Commercial ASIC Flows

Status: Architecture / design note

Primary implementation target: Tiny Tapeout ASIC flows, initially around the current Tiny Tapeout node and its LibreLane-based implementation flow

Future direction: Optional adapters for commercial EDA environments such as Cadence and Synopsys, without making those tools a requirement for the initial library

1. Purpose

`hardcaml_asic` should provide a common OCaml/Hardcaml-facing abstraction layer for ASIC-oriented design concerns that do not naturally belong in ordinary RTL construction.

The immediate motivation is practical: make Hardcaml designs easier to target at Tiny Tapeout and similar open ASIC flows. Examples include:

- instantiating SRAM and other technology-specific macros through a common interface;
- describing ASIC-specific implementation intent without scattering PDK-specific names and file paths throughout RTL;
- generating or coordinating the configuration required by Tiny Tapeout and LibreLane flows;
- exposing implementation results in a structured way that OCaml programs, CI systems, or a future RTL workbench can consume.

The library should be designed so that a later commercial-tooling layer is possible, but **commercial Cadence/Synopsys support is not an initial implementation requirement**.

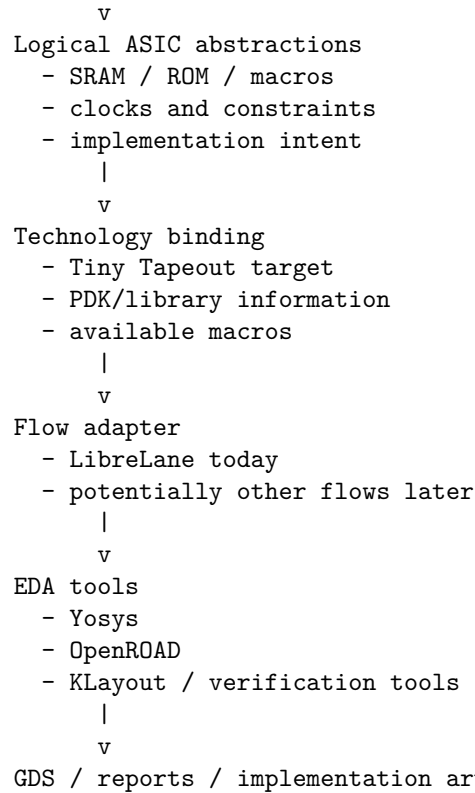
The design objective is therefore:

Build the right abstractions for Tiny Tapeout now, while avoiding assumptions that would make future industrial ASIC-flow adapters unnecessarily difficult.

2. Mental Model

A useful decomposition is:

Hardcaml design
|



The important architectural decision is that normal Hardcaml modules should describe **what hardware capability they need**, rather than hard-code the exact PDK cell or EDA command required to implement it.

3. PDK, Standard Cells, and Tooling Are Separate Concepts

`hardcaml_asic` should keep a clean conceptual distinction between the following layers.

3.1 PDK

The process design kit describes the fabrication technology. It contains information such as:

- process layers and routing stack;
- transistor and passive-device models;
- physical design rules;
- via definitions;

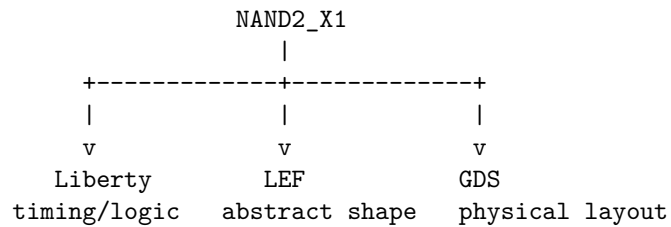
- parasitic models;
- DRC/LVS decks and associated technology data.

3.2 Standard-cell and macro libraries

Libraries built for the PDK provide reusable implementation blocks such as:

- inverters, NAND/NOR gates, AOI/OAI gates;
- flip-flops and latches;
- buffers and clock cells;
- SRAM or ROM macros;
- IO cells and other hard macros.

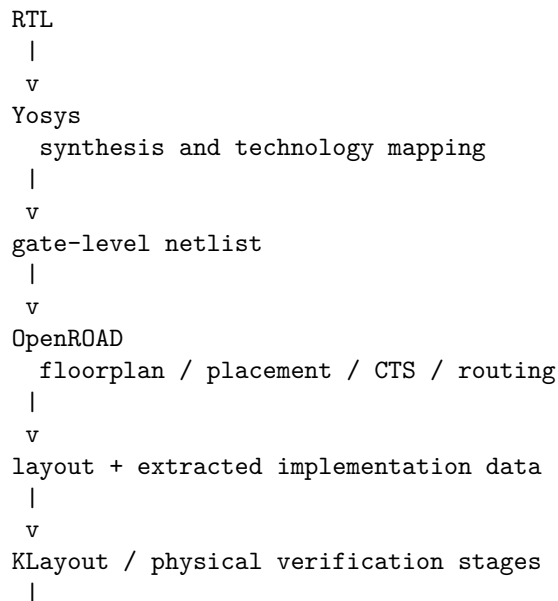
A cell may have multiple views of the same underlying object:



3.3 EDA flow

The EDA flow consumes RTL plus PDK/library information and produces the physical implementation.

For the open flow, a simplified picture is:



v
GDS

LibreLane coordinates these tools and the files passed between them.

4. LibreLane's Role

For architectural purposes, LibreLane can be treated as the **flow owner/orchestrator** for the current open ASIC target.

A useful FPGA analogy is:

FPGA world	Open ASIC world
-----	-----
Vivado synth	Yosys
Vivado implementation	OpenROAD
Vivado overall flow ownership	LibreLane + underlying tools

The analogy is not exact: Vivado contains most of its own implementation engines, whereas LibreLane primarily coordinates independent tools.

`hardcaml_asic` therefore should not attempt to replace LibreLane. It should make it easier to **describe, configure, invoke, and consume** such flows from OCaml.

5. Initial Scope: Tiny Tapeout First

The first useful version of `hardcaml_asic` should focus on concrete needs encountered when targeting Tiny Tapeout.

Possible initial responsibilities include:

1. **Technology-neutral macro requests**
 - SRAM
 - ROM
 - black-box hard macros
 - possibly simple clock/reset infrastructure where appropriate
2. **Tiny Tapeout technology bindings**
 - map logical requests to macros actually available on the target node;
 - express unsupported configurations clearly;
 - keep exact PDK names out of ordinary user RTL where possible.
3. **LibreLane/Tiny Tapeout flow integration**
 - generate required project/configuration files;
 - identify RTL sources and top-level modules;
 - pass clocks and other implementation constraints;
 - invoke the existing flow rather than reimplement it.

4. Artifact and report handling

- locate generated netlists, DEF/GDS, timing reports, area reports, and logs;
- provide a typed result to the rest of the OCaml program.

5. Simulation fallbacks

- provide behavioral models for technology-specific macros where practical;
- permit Hardcaml simulations without requiring an ASIC flow to be installed.

The goal is not to invent a universal ASIC framework before the Tiny Tapeout use case works well.

6. SRAM as the Model Abstraction

SRAM is a good example of the abstraction style the library should encourage.

A design should ideally be able to request something conceptually like:

```
Sram.create
~depth:1024
~width:64
~ports:`One_rw
```

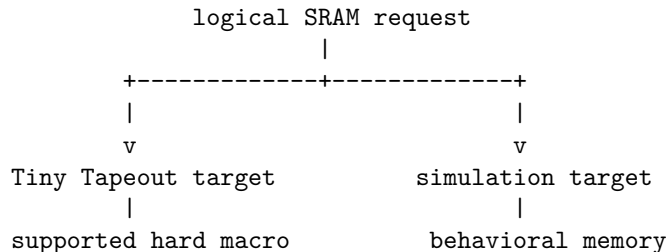
rather than directly instantiate a name such as:

```
some_pdk_specific_sram_1024x64_1rw
```

The request describes logical requirements:

```
1024 words
64 bits per word
one read/write port
possibly byte enables
specified read latency / semantics
```

A technology binding then decides how that request is implemented:



In a future proprietary process, the same request might map to a foundry or memory-compiler-generated macro.

This separation is valuable even if commercial flows are never implemented: it makes the Tiny Tapeout code cleaner and isolates technology-specific details.

7. Proposed Layering

A reasonable long-term module structure is:

```
hardcam1_asic
|
+-- Macro
|   +-- Sram
|   +-- Rom
|   +-- Black_box
|   `-- Custom_macro
|
+-- Technology
|   +-- macro capabilities
|   +-- standard-cell / library metadata
|   +-- process corners
|   +-- physical views
|   `-- target-specific bindings
|
+-- Constraint
|   +-- clocks
|   +-- IO delays
|   +-- false paths
|   +-- multicycle paths
|   `-- clock uncertainty
|
+-- Flow
|   +-- project description
|   +-- synthesis intent
|   +-- implementation intent
|   +-- artifact graph
|   `-- execution
|
+-- Tool
|   +-- LibreLane
|   +-- Yosys                [possibly internal/detail-level]
|   +-- OpenROAD             [possibly internal/detail-level]
|   `-- future adapters
|
`-- Report
    +-- Timing
    +-- Area
```

```
+++ Power
+++ Utilization
`-- Diagnostics
```

This does **not** mean every module needs to exist in version 1. It is a direction for keeping boundaries clear as the project grows.

8. Constraints as Typed Intent

Another useful abstraction is timing/implementation constraints.

Instead of forcing design code to emit raw SDC everywhere, `hardcaml_asic` could eventually expose typed OCaml constructs such as:

```
Constraint.Clock.create
  ~name:"core_clk"
  ~period:(Time.ns 1.0)
  clk
```

A backend can then serialize this into the representation expected by the active flow, typically SDC or Tcl.

The advantage is not merely syntax. Typed constraints can:

- validate units;
- associate constraints with Hardcaml ports/signals;
- make flow generation reproducible;
- permit future backend changes without rewriting design intent;
- provide structured information to visualization or analysis tools.

Raw SDC should still be available as an escape hatch.

9. Tool Adapters Should Be Process-Oriented, Not FFI-Oriented

A future commercial-tool integration should probably **not** begin as C bindings into Cadence or Synopsys internals.

Most industrial EDA tools are naturally driven through batch commands and Tcl scripts. A practical adapter architecture is therefore:

```
OCaml
|
|
|+++ generate Tcl / SDC / manifests / config
|
|+++ launch external EDA executable
|
```

```

+-- monitor process and collect logs
|
+-- collect output artifacts
|
+-- parse reports into typed OCaml values

```

For example, a synthesis backend could eventually follow an interface resembling:

```

module type Synthesis_tool = sig
  type config

  val synthesize :
    config ->
    rtl:File.t list ->
    constraints:Constraint.t list ->
    output:File.t ->
    Result.t
end

```

Possible implementations could be:

```

module Yosys : Synthesis_tool

(* Future, optional adapters *)
module Genus : Synthesis_tool
module Design_compiler : Synthesis_tool

```

The first implementation should remain focused on the open flow.

10. Future Commercial EDA Support

Commercial support should be treated as an **additional layer down the road**, not as a requirement for the Tiny Tapeout implementation.

The architecture can nonetheless avoid choices that would make this future direction impossible.

Potential future adapters include:

```

Cadence
  Genus      - synthesis
  Innovus    - physical implementation
  Tempus     - timing signoff
  Quantus    - extraction
  Voltus     - power / IR analysis

```

Synopsys

Design Compiler / Fusion Compiler	- synthesis / implementation
IC Compiler II	- physical implementation
PrimeTime	- timing signoff
StarRC	- extraction

The important point is that supporting these tools is not just a matter of changing an executable path. Tool invocation itself is easy; representing the surrounding ASIC environment is harder.

11. Why `install_location` Alone Is Not Enough

A simplistic adapter might look like:

```
type tool =
  { install_directory : string }
```

That is unlikely to be sufficient in a real industrial environment.

A tool may depend on:

- a particular executable version;
- license-server configuration;
- environment variables;
- shell/module setup;
- site-specific initialization Tcl;
- PDK/library locations;
- compute-farm or scheduler wrappers;
- company-owned launch scripts.

A more flexible process layer might eventually support concepts such as:

```
type executable =
  | Path of string
  | Command of string

type launcher =
  | Direct
  | Shell_wrapper of string
  | Environment_module of string
  | Slurm of ...
  | Lsf of ...
```

This need not be implemented now. The important architectural lesson is simply:

Do not bake the assumption that every EDA tool is a binary at a fixed absolute path directly into higher-level flow APIs.

For the current Tiny Tapeout implementation, a much smaller execution model is sufficient.

12. The Hard Part of Commercial Support Is Flow Semantics

Launching `genus`, `innovus`, or `dc_shell` is relatively straightforward. The difficult problem is expressing the design environment they consume.

A serious commercial flow can involve:

```
RTL
|
+-- SDC timing constraints
+-- standard-cell libraries
|   +-- multiple voltage / temperature / process corners
|
+-- physical libraries
|   +-- LEF
|   +-- GDS / vendor database formats
|
+-- RC technology files
+-- SRAM / PLL / IO macro libraries
+-- power intent such as UPF
+-- multiple modes and multiple corners
+-- floorplan constraints
+-- pin constraints
+-- power-grid strategy
+-- CTS strategy
+-- routing constraints
+-- tool-specific optimization settings
```

A future `hardcaml_asic` commercial layer therefore becomes interesting not because it can run an executable, but because it can provide a coherent, typed model of this implementation intent.

This is also why the initial Tiny Tapeout implementation should remain modest: attempting to fully model industrial MMMC, power intent, signoff, DFT, and proprietary databases before the open-flow API is proven would likely overcomplicate the project.

13. Avoid the Lowest-Common-Denominator Trap

A future multi-tool API should not pretend that OpenROAD, Innovus, ICC2, and Fusion Compiler all have exactly the same capabilities.

The desirable structure is:

```
portable high-level API
+
backend-specific extensions / escape hatches
```

For example, a common flow could expose portable concepts such as:

- target utilization;
- clock definitions;
- die/core areas;
- macro placement constraints;
- timing goals.

A future Cadence backend should still permit raw or structured Cadence-specific extensions when an advanced Innovus feature is needed.

Conceptually:

```
Flow.vendor_extension
(Innovus (...))
```

or, at the lowest level:

```
Innovus.raw_tcl "set_db ..."
```

The abstraction should simplify common work, not make advanced work impossible.

14. Normalized Reports May Be One of the Most Valuable Features

A particularly useful cross-flow abstraction is a normalized result/report API.

Different tools emit timing, area, power, and utilization information in different formats. A typed OCaml representation could allow users to write:

```
let%bind result = Flow.run flow in

printf "WNS: %s\n"
  (Time.to_string result.timing.wns);

printf "Area: %s\n"
  (Area.to_string result.area)
```

without caring whether the underlying data came from OpenROAD today or a commercial signoff tool in the future.

Potential normalized fields include:

- WNS and TNS;

- critical paths;
- cell count;
- combinational/sequential area;
- total core area;
- utilization;
- macro inventory;
- clock information;
- power, when available;
- DRC/error counts;
- congestion metrics;
- generated artifact paths.

This would also make `hardcaml_asic` a natural backend for CI dashboards or a future Hardcaml RTL workbench.

15. Relationship to a Future RTL Workbench

A flow/result API could eventually support a higher-level interface such as:

```
+-----+
| Hardcaml ASIC Run                               |
|                                                  |
| Target:      Tiny Tapeout / SG13...           |
| Backend:     LibreLane                         |
|                                                  |
| WNS:         ...                               |
| Area:        ...                               |
| Utilization: ...                               |
| Cells:       ...                               |
|                                                  |
| [Timing] [Floorplan] [Logs] [Artifacts]        |
+-----+
```

Later, a commercial backend could populate the same high-level UI while preserving access to tool-specific details.

This possibility is another reason to return structured results rather than only shell exit codes and text logs.

16. Recommended Development Phases

Phase 1 - Tiny Tapeout fundamentals

Focus exclusively on making the current target pleasant and reliable.

- establish core package/module structure;

- implement SRAM/macro abstraction needed by real TT projects;
- provide simulation behavior where useful;
- integrate with the chosen Tiny Tapeout/LibreLane flow;
- generate deterministic project/configuration artifacts;
- collect useful output artifacts and errors.

Phase 2 - Stabilize the technology and flow abstractions

After real projects have exercised Phase 1:

- separate logical macro requirements from technology bindings;
- formalize typed clocks/constraints where this proves useful;
- define a stable `Technology.t` / target interface;
- normalize timing/area/utilization reports;
- improve flow graph/artifact tracking.

Phase 3 - Additional open processes/flows

Use another open target to verify that abstractions are actually portable rather than merely renamed Tiny Tapeout assumptions.

This is an important architectural test before designing commercial adapters.

Phase 4 - Optional commercial tooling adapters

Only after the common API is mature:

- add environment/executable discovery abstractions;
- prototype one narrow commercial integration, such as synthesis-only Genus or Design Compiler support;
- generate Tcl from common constraints/flow intent;
- parse reports into the existing result representation;
- retain backend-specific escape hatches.

Commercial EDA support should remain optional and should not force proprietary software, files, or licenses on users of the open-source Tiny Tapeout path.

17. Suggested Repository Direction

One possible future organization is:

```
hardcaml_asic/
|
+-- src/
|   +-- macro/
|       |   +-- sram.ml
|       |   +-- rom.ml
|       |   `-- black_box.ml
```

```

|   |
|   +-- technology/
|   |   +-- technology.ml
|   |   |-- tiny_tapeout_*.ml
|   |
|   +-- constraint/
|   |   +-- clock.ml
|   |   |-- constraint.ml
|   |
|   +-- flow/
|   |   +-- flow.ml
|   |   +-- artifact.ml
|   |   |-- result.ml
|   |
|   +-- tool/
|   |   |-- librelane.ml
|   |
|   |-- report/
|   |   +-- timing.ml
|   |   |-- area.ml
|   |
+-- examples/
|   |-- tiny_tapeout/
|
|-- test/

```

The exact hierarchy should follow what the first implementation actually needs; empty architectural layers should not be created merely for symmetry.

18. Design Principles

The project should favor the following principles.

Tiny Tapeout is the product target now

Do not delay a useful TT implementation in order to design a hypothetical universal interface for tools that are not currently available or required.

Extensibility should shape boundaries, not inflate scope

It is worth separating concepts such as logical SRAM requests, technology bindings, and flow execution now because those boundaries improve the current design as well as future portability.

It is not worth implementing industrial MMMC, DFT, UPF, or signoff abstractions merely because a commercial flow may need them someday.

Keep technology names out of design logic where practical

Hardcaml modules should describe hardware behavior and capability requirements. PDK-specific macro names and physical files belong in target-specific bindings.

Treat external EDA tools as processes

Generate their scripts and inputs, invoke them, collect their outputs, and parse their reports. Avoid unnecessary dependencies on private APIs.

Prefer typed intent, but preserve escape hatches

Typed OCaml representations are valuable for validation, refactoring, and tooling. Raw Tcl/SDC/configuration hooks remain necessary for unusual or backend-specific requirements.

Normalize outputs before trying to normalize every input

A stable `Result.t` for timing, area, artifacts, and diagnostics can provide immediate value and is often easier to make portable than a universal representation of every possible implementation knob.

Let the abstractions emerge from real designs

The Tiny Tapeout implementation should be the proving ground. Generalize only after concrete examples reveal which concepts are truly common.

19. Summary

The near-term vision for `hardcaml_asic` is not to reproduce an industrial ASIC CAD stack in OCaml. It is to provide the missing layer between Hardcaml designs and practical Tiny Tapeout ASIC implementation:

```
Hardcaml
  |
  v
hardcaml_asic
  |
  +-- technology-neutral macro abstractions
  +-- Tiny Tapeout technology bindings
  +-- constraints / implementation intent where useful
  +-- LibreLane flow integration
  `-- structured artifacts and reports
  |
  v
```

```

Tiny Tapeout / LibreLane
|
v
Yosys + OpenROAD + physical verification
|
v
GDS

```

The architecture should, however, leave a credible path toward:

```

hardcaml_asic
|
+-- open-flow adapters
|
`-- optional future commercial adapters
    +-- Cadence
    `-- Synopsys

```

Commercial EDA support is therefore best viewed as a **future layer enabled by good boundaries today**, rather than a current feature requirement.

The most important architectural bets are:

1. represent logical macro requirements separately from PDK-specific implementations;
2. keep flow/tool invocation behind explicit adapters;
3. avoid assuming tools live at fixed paths or have identical capabilities;
4. return structured reports and artifacts to OCaml;
5. preserve backend-specific escape hatches;
6. allow the Tiny Tapeout implementation to drive and validate the abstractions before expanding outward.

That gives `hardcaml_asic` a focused immediate mission while leaving room to grow into a broader Hardcaml-oriented ASIC implementation interface if real use cases justify it.